

NAME - ADITYA KUMAR

UNIVERSITY ROLL NUMBER - 34230821034

Course Name : Soft Computing Lab

Course Code: PECAIML693

3rd Year 6th Semester

QUESTION 1

Implement Union, Intersection, Complement, Algebraic Difference, Algebraic product, Multiplication of fuzzy set with crisp number, Power of fuzzy set, Algebraic sum, Bounded sum, Bounded difference and Cartesian product on fuzzy sets.

```
1 def union(a,b):
2     res=[max(a[i],b[i]) for i in range(len(a))]
3     return res
4 a=[0.3,0.8,0.1]
5 b=[0.5,0.7,0.2]
6 print("UNION OF a, b",union(a,b))
```

UNION OF a, b [0.5, 0.8, 0.2]

```
1 def intersection(a,b):
2     res=[min(a[i],b[i]) for i in range(len(a))]
3     return res
4 a=[0.3,0.8,0.1]
5 b=[0.5,0.7,0.2]
6 print("INTERSECTION OF a, b",intersection(a,b))
```

INTERSECTION OF a, b [0.3, 0.7, 0.1]

```
1 def complement(a):
2     res=[x:=float(1-a[i]) for i in range(len(a))]
3     return res
4 a=[0.3,0.6,0.1]
5 print("Complement of a",complement(a))
6
```

Complement of a [0.7, 0.4, 0.9]

```
1 def fuzzy_sum(a,b):
2     res=[(a[i]+b[i])-(a[i]*b[i]) for i in range(len(a))]
3     return res
4 a=[0.3,0.5,0.1]
5 b=[0.5,0.4,0.2]
6 print("FUZZY SUM OF a, b",fuzzy_sum(a,b))
```

FUZZY SUM OF a, b [0.65, 0.7, 0.28]

```
1 def fuzzy_product(a,b):
2     res=[a[i]*b[i] for i in range(len(a))]
3     return res
4 a=[0.3,0.5,0.5]
5 b=[0.5,0.4,0.4]
6 print("FUZZY PRODUCT OF a, b", fuzzy_product(a,b))
```

FUZZY PRODUCT OF a, b [0.15, 0.2, 0.2]

```
1 def fuzzy_difference(a,b):
2     res=[max((a[i]-b[i]),0) for i in range(len(a))]
3     return res
4 a=[0.2,0.8,0.1]
5 b=[0.5,0.4,0.2]
6 print("FUZZY DIFFERENCE OF a,b",fuzzy_difference(a,b))
7
```

FUZZY DIFFERENCE OF a,b [0, 0.4, 0]

```

1 def scalar_product(alpha,a):
2     res=[alpha*a[i] for i in range(len(a))]
3     return res
4 a=[0.2,0.8,0.1]
5 print("SCALAR PRODUCT of a,b",scalar_product(2,a))
6

```

SCALAR PRODUCT of a,b [0.4, 1.6, 0.2]

```

1 def difference(a,b):
2     res=[(intersection(a,complement(b)))]
3     return res
4 a=[0.2,0.8,0.1]
5 b=[0.5,0.4,0.2]
6 print("DIFFERENCE of a,b",difference(a,b))
7

```

DIFFERENCE of a,b [[0.2, 0.6, 0.1]]

```

1 def dis_sum(a,b):
2     res=[union((intersection(complement(a),b)),intersection(a,complement(b)))]
3     return res
4 a=[0.2,0.8,0.1]
5 b=[0.5,0.4,0.2]
6 print(dis_sum(a,b))
7

```

[[0.5, 0.6, 0.2]]

```

1 def bound_diff(a,b):
2     res=[max(0,((a[i]+b[i])-1)) for i in range(len(a))]
3     return res
4 a=[0.2,0.8,0.1]
5 b=[0.5,0.4,0.2]
6 print("BOUND DIFFERENCE of a,b",bound_diff(a,b))
7

```

BOUND DIFFERENCE of a,b [0, 0.2000000000000018, 0]

```

1 def bound_sum(a,b):
2     res=[min(1,((a[i]+b[i]))) for i in range(len(a))]
3     return res
4 a=[0.2,0.8,0.1]
5 b=[0.5,0.4,0.2]
6 print("BOUND SUM of a,b",bound_sum(a,b))
7

```

BOUND SUM of a,b [0.7, 1, 0.3000000000000004]

```

1 def equ(a,b):
2     return a==b
3 a=[0.5,0.4,0.2]
4 b=[0.5,0.4,0.2]
5 print(equ(a,b))
6

```

True

```

1 import math;
2 def printPowerSet(a):
3     # with set_size n is (2**n -1)
4     pow_set_size = (int) (math.pow(2,len(a)));
5     counter = 0;
6     j = 0;
7     # Run from counter 000..0 to 111..1
8     for counter in range(0, pow_set_size):
9         for j in range(0, len(a)):
10             if((counter & (1 << j)) > 0):
11                 print(set[j], end = "");
12             print("");
13 # Driver program to test printPowerSet
14 b=[0.5,0.4,0.2]
15 printPowerSet(b)

```

```

set[0]
set[1]
set[0]set[1]
set[2]

```

```

set[0]set[2]
set[1]set[2]
set[0]set[1]set[2]

1 import itertools
2 def power_set(s):
3     return list(itertools.chain.from_iterable(itertools.combinations(s, r) for r in range(len(s))))
4 my_set = {0.5,0.4,0.2}
5 my_power_set = power_set(my_set)
6 print(my_power_set)

[(), (0.5,), (0.4,), (0.2,), (0.5, 0.4), (0.5, 0.2), (0.4, 0.2)]

1 Start coding or generate with AI.

1 import numpy as np
2 def cartesion_product(a,b):
3     k=[]
4     for i in range(len(a)):
5         res=[min(a[i],b[j]) for j in range(len(b))]
6         k.append(res)
7     return k
8 a=[0.2,0.3,0.5,0.6]
9 b=[0.8,0.6,0.3]
10 #print(np.array(cartesion_product(a,b)))
11 z=cartesion_product(a,b)
12 for i in z:
13     print(i)
14

[0.2, 0.2, 0.2]
[0.3, 0.3, 0.3]
[0.5, 0.5, 0.3]
[0.6, 0.6, 0.3]

1 def fuzzification(s):
2     a=s[0]
3     b=s[int((len(s))/2)]
4     c=s[-1]
5     k=[]
6     for i in range(len(s)):
7         if s[i]<=a:
8             z=0
9         elif s[i]>=a and s[i]<=b:
10            z=(s[i]-a)/(b-a)
11        elif s[i]>=b and s[i]<=c:
12            z=(c-s[i])/(c-b)
13        else:
14            z=0
15        k.append([s[i],z])
16    return k
17 a=[10,20,30,40,50,60,70,80]
18 print("FUZZIFICATION :",fuzzification(a))

FUZZIFICATION : [[10, 0], [20, 0.25], [30, 0.5], [40, 0.75], [50, 1.0], [60, 0.6666666666666666], [70, 0.3333333333333333], [80, 0.0]]

```

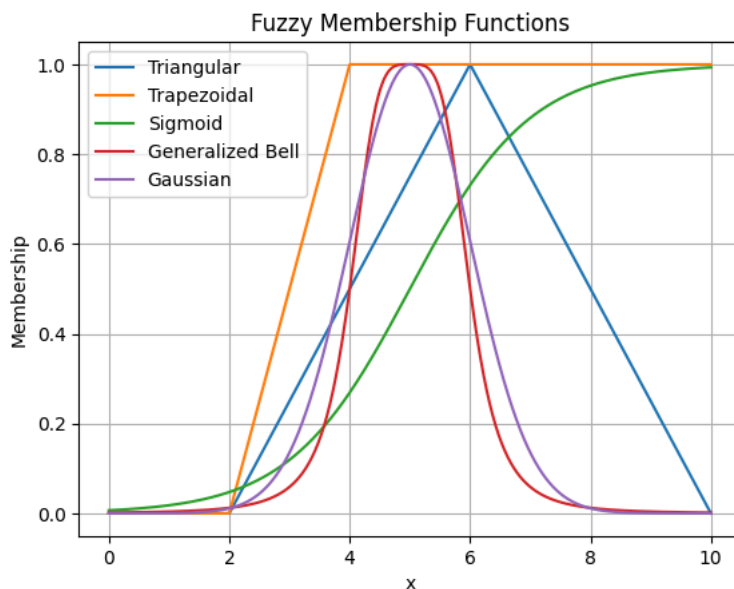
QUESTION 2

Realization and plotting of different fuzzy membership function: Triangular Membership function, Trapezoidal membership function, Sigmoid membership function, Generalized bell membership function and Gaussian membership function.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Triangular Membership Function
5 def triangular(x, a, b, c):
6     return np.maximum(0, np.minimum((x - a) / (b - a), (c - x) / (c - b)))
7
8 # Trapezoidal Membership Function
9 def trapezoidal(x, a, b, c, d):
10    return np.maximum(0, np.minimum(np.minimum((x - a) / (b - a), 1), (d - x) / (d - c)), 1))
11
12 # Sigmoid Membership Function
13 def sigmoid(x, a, b):
14    return 1 / (1 + np.exp(-a*(x-b)))
15
16 # Generalized Bell Membership Function
17 def bell(x, a, b, c):
18    return 1 / (1 + np.abs((x - c) / a) ** (2 * b))
19
20 # Gaussian Membership Function
21 def gaussian(x, mean, sigma):
22    return np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))
23
24 # Plotting the membership functions
25 x = np.linspace(0, 10, 1000)
26
27 # Triangular
28 plt.plot(x, triangular(x, 2, 6, 10), label='Triangular')
29 # Trapezoidal
30 plt.plot(x, trapezoidal(x, 2, 4, 8, 10), label='Trapezoidal')
31 # Sigmoid
32 plt.plot(x, sigmoid(x, 1, 5), label='Sigmoid')
33 # Generalized Bell
34 plt.plot(x, bell(x, 1, 2, 5), label='Generalized Bell')
35 # Gaussian
36 plt.plot(x, gaussian(x, 5, 1), label='Gaussian')
37
38 plt.title('Fuzzy Membership Functions')
39 plt.xlabel('x')
40 plt.ylabel('Membership')
41 plt.legend()
42 plt.grid(True)
43 plt.show()
44

```



QUESTION 3

Implement fuzzification on crisp set using following membership functions: i. Triangular Membership function ii. Trapezoidal membership function iii. Sigmoid membership function iv. Generalized bell membership function v. Gaussian membership function

```

1 import numpy as np
2
3 # Triangular Membership function
4 def triangular(x, a, b, c):
5     if a <= x < b:
6         return (x - a) / (b - a)
7     elif b <= x < c:
8         return (c - x) / (c - b)
9     else:
10        return 0
11
12 # Trapezoidal membership function
13 def trapezoidal(x, a, b, c, d):
14     if a <= x < b:
15         return (x - a) / (b - a)
16     elif b <= x <= c:
17         return 1
18     elif c < x <= d:
19         return (d - x) / (d - c)
20     else:
21         return 0
22
23 # Sigmoid membership function
24 def sigmoid(x, a, c):
25     return 1 / (1 + np.exp(-a * (x - c)))
26
27 # Generalized bell membership function
28 def generalized_bell(x, a, b, c):
29     return 1 / (1 + np.abs((x - c) / a) ** (2 * b))
30
31 # Gaussian membership function
32 def gaussian(x, mean, sigma):
33     return np.exp(-0.5 * ((x - mean) / sigma) ** 2)
34
35 # Crisp input
36 crisp_value = 0.7
37
38 # Fuzzification using various membership functions
39 triangular_membership = triangular(crisp_value, 0.2, 0.5, 0.8)
40 trapezoidal_membership = trapezoidal(crisp_value, 0.2, 0.4, 0.6, 0.8)
41 sigmoid_membership = sigmoid(crisp_value, 2, 0.5)
42 bell_membership = bell(crisp_value, 1, 2, 0.5)
43 gaussian_membership = gaussian(crisp_value, 0.5, 0.1)
44
45 print("Fuzzification using various membership functions:")
46 print("Triangular Membership:", triangular_membership)
47 print("Trapezoidal Membership:", trapezoidal_membership)
48 print("Sigmoid Membership:", sigmoid_membership)
49 print("Generalized Bell Membership:", bell_membership)
50 print("Gaussian Membership:", gaussian_membership)
51
52
53 Fuzzification using various membership functions:
54 Triangular Membership: 0.3333333333333336
55 Trapezoidal Membership: 0.5000000000000002
56 Sigmoid Membership: 0.598687660112452
57 Generalized Bell Membership: 0.998402555910543
58 Gaussian Membership: 0.1353352832366128

```

QUESTION 4

Realization of characteristics of fuzzy set by finding the following values: i. Core of a fuzzy set ii. Support of a fuzzy set. iii. Crossover points of a fuzzy set. iv. Bandwidth of a fuzzy set. v. Convexity of a fuzzy set.

```

1 import numpy as np
2
3 # Core of a fuzzy set
4 def core(fuzzy_set, threshold=0.5):
5     return [x for x, y in enumerate(fuzzy_set) if y >= threshold]
6
7 # Support of a fuzzy set
8 def support(fuzzy_set, threshold=0.5):
9     return [x for x, y in enumerate(fuzzy_set) if y > threshold]
10
11 # Crossover points of a fuzzy set
12 def crossover_points(fuzzy_set, threshold=0.5):
13     points = []
14     for i in range(len(fuzzy_set) - 1):
15         if fuzzy_set[i] <= threshold < fuzzy_set[i + 1]:
16             points.append(i + (threshold - fuzzy_set[i]) / (fuzzy_set[i + 1] - fuzzy_set[i]))
17         elif fuzzy_set[i] > threshold >= fuzzy_set[i + 1]:
18             points.append(i + 1 - (fuzzy_set[i] - threshold) / (fuzzy_set[i] - fuzzy_set[i + 1]))
19     return points
20
21 # Bandwidth of a fuzzy set
22 def bandwidth(fuzzy_set):
23     return len(fuzzy_set) - 1
24
25 # Convexity of a fuzzy set
26 def convexity(fuzzy_set):
27     if all(fuzzy_set[i] <= fuzzy_set[i + 1] for i in range(len(fuzzy_set) - 1)):
28         return "Convex"
29     elif all(fuzzy_set[i] >= fuzzy_set[i + 1] for i in range(len(fuzzy_set) - 1)):
30         return "Concave"
31     else:
32         return "Mixed"
33
34 # Example fuzzy set
35 fuzzy_set = [0.1, 0.3, 0.7, 1, 0.8, 0.5]
36
37 # Core of the fuzzy set
38 print("Core of the fuzzy set:", core(fuzzy_set))
39
40 # Support of the fuzzy set
41 print("Support of the fuzzy set:", support(fuzzy_set))
42
43 # Crossover points of the fuzzy set
44 print("Crossover points of the fuzzy set:", crossover_points(fuzzy_set))
45
46 # Bandwidth of the fuzzy set
47 print("Bandwidth of the fuzzy set:", bandwidth(fuzzy_set))
48
49 # Convexity of the fuzzy set
50 print("Convexity of the fuzzy set:", convexity(fuzzy_set))
51

```

```

Core of the fuzzy set: [2, 3, 4, 5]
Support of the fuzzy set: [2, 3, 4]
Crossover points of the fuzzy set: [1.5, 4.0]
Bandwidth of the fuzzy set: 5
Convexity of the fuzzy set: Mixed

```

QUESTION 5

Solving some real-world problem using fuzzy set and fuzzy proposition.

Consider a real-world problem: Traffic Light Control System.

- Problem Statement:** We want to optimize the traffic flow at an intersection controlled by traffic lights. Traditional traffic light systems work with fixed timings, which may not be efficient, especially during varying traffic conditions. Using fuzzy logic, we can develop a dynamic traffic light control system that adjusts the signal timings based on real-time traffic conditions.
- Solution Approach:** We can use fuzzy logic to determine the optimal timing for each traffic light phase based on factors such as traffic density, vehicle queue length, and waiting time. By considering these factors as inputs to our fuzzy system, we can dynamically adjust the signal timings to minimize congestion and waiting times.

- Fuzzy Sets:**

Traffic Density: We can define fuzzy sets such as "low," "medium," and "high" to represent the density of vehicles approaching the intersection.

Vehicle Queue Length: We can define fuzzy sets such as "short," "medium," and "long" to represent the length of the vehicle queues at each approach.

Waiting Time: We can define fuzzy sets such as "short," "medium," and "long" to represent the waiting time experienced by vehicles at the intersection.

- **Fuzzy Propositions:** We can create fuzzy rules to control the traffic lights based on the inputs mentioned above. For example:

IF traffic density is high AND vehicle queue length is long THEN increase green time for that direction. IF waiting time is long THEN decrease green time for other directions to alleviate congestion. IF traffic density is low AND vehicle queue length is short THEN decrease green time for that direction.

- **Implementation:** We can implement this solution using a fuzzy inference system. We'll define membership functions for each input variable, fuzzy rules based on these sets, and an inference engine to determine the optimal signal timings for each phase of the traffic lights.

```
1 !pip install scikit-fuzzy
```

```
2
```

```
Collecting scikit-fuzzy
```

```
  Downloading scikit-fuzzy-0.4.2.tar.gz (993 kB)
```

```
    994.0/994.0 kB 9.6 MB/s eta 0:00:00
```

```
  Preparing metadata (setup.py) ... done
```

```
Requirement already satisfied: numpy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-fuzzy) (1.25.2)
```

```
Requirement already satisfied: scipy>=0.9.0 in /usr/local/lib/python3.10/dist-packages (from scikit-fuzzy) (1.11.4)
```

```
Requirement already satisfied: networkx>=1.9.0 in /usr/local/lib/python3.10/dist-packages (from scikit-fuzzy) (3.3)
```

```
Building wheels for collected packages: scikit-fuzzy
```

```
  Building wheel for scikit-fuzzy (setup.py) ... done
```

```
    Created wheel for scikit-fuzzy: filename=scikit_fuzzy-0.4.2-py3-none-any.whl size=894078 sha256=8fe327559ffa3cf35f2b5e5215b450f18c
```

```
    Stored in directory: /root/.cache/pip/wheels/4f/86/1b/dfd97134a2c8313e519bcebd95d3fedc7be7944db022094bc8
```

```
Successfully built scikit-fuzzy
```

```
Installing collected packages: scikit-fuzzy
```

```
Successfully installed scikit-fuzzy-0.4.2
```

```

1 ##### A real-world problem: Traffic Light Control System #####
2
3 #IMPORTING NECESSARY LIBRARIES
4
5 import numpy as np
6 import skfuzzy as fuzz
7 from skfuzzy import control as ctrl
8
9 # Antecedents (input variables)
10 traffic_density = ctrl.Antecedent(np.arange(0, 11, 1), 'traffic density')
11 queue_length = ctrl.Antecedent(np.arange(0, 11, 1), 'queue length')
12 waiting_time = ctrl.Antecedent(np.arange(0, 11, 1), 'waiting time')
13
14 # Consequents (output variables)
15 green_time_North = ctrl.Consequent(np.arange(0, 31, 1), 'green time North')
16 green_time_South = ctrl.Consequent(np.arange(0, 31, 1), 'green time South')
17 green_time_East = ctrl.Consequent(np.arange(0, 31, 1), 'green time East')
18 green_time_West = ctrl.Consequent(np.arange(0, 31, 1), 'green time West')
19
20 # Fuzzy sets for input variables
21 traffic_density.automf(3, names=['poor', 'average', 'good'])
22 queue_length.automf(3, names=['short', 'medium', 'long'])
23 waiting_time.automf(3, names=['short', 'medium', 'long'])
24
25 # Fuzzy sets for output variables
26 green_time_North.automf(3, names=['short', 'medium', 'long'])
27 green_time_South.automf(3, names=['short', 'medium', 'long'])
28 green_time_East.automf(3, names=['short', 'medium', 'long'])
29 green_time_West.automf(3, names=['short', 'medium', 'long'])
30
31 # Define fuzzy rules
32 rule1 = ctrl.Rule(traffic_density['good'] & queue_length['long'], green_time_North['long'])
33 rule2 = ctrl.Rule(traffic_density['good'] & queue_length['long'], green_time_South['long'])
34 rule3 = ctrl.Rule(waiting_time['long'], green_time_North['short'])
35 rule4 = ctrl.Rule(waiting_time['long'], green_time_South['short'])
36 rule5 = ctrl.Rule(traffic_density['poor'] & queue_length['short'], green_time_North['medium'])
37 rule6 = ctrl.Rule(traffic_density['poor'] & queue_length['short'], green_time_South['medium'])
38 rule7 = ctrl.Rule(traffic_density['good'] & queue_length['long'], green_time_East['long'])
39 rule8 = ctrl.Rule(traffic_density['good'] & queue_length['long'], green_time_West['long'])
40
41 # Control system
42 traffic_light_control = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6, rule7, rule8])
43 traffic_light = ctrl.ControlSystemSimulation(traffic_light_control)
44
45 # Set inputs
46 traffic_light.input['traffic density'] = 7
47 traffic_light.input['queue length'] = 8
48 traffic_light.input['waiting time'] = 6
49
50 # Compute output
51 traffic_light.compute()
52
53 # Output result
54 print("Green Time North:", traffic_light.output['green time North'])
55 print("Green Time South:", traffic_light.output['green time South'])
56 print("Green Time East:", traffic_light.output['green time East'])
57 print("Green Time West:", traffic_light.output['green time West'])
58
59
60 Green Time North: 17.72
61 Green Time South: 17.72
62 Green Time East: 23.875000000000004
63 Green Time West: 23.875000000000004

```

QUESTION 6

Create fuzzy relation by Cartesian product of any two fuzzy sets and perform max-min and max-product composition on any two fuzzy relations.

```

1 import numpy as np
2 import skfuzzy as fuzz
3
4 # Define universe and membership functions for fuzzy sets A and B
5 x = np.arange(0, 11, 1)
6 A = fuzz.trimf(x, [3, 6, 9]) # Triangular membership function for fuzzy set A
7 B = fuzz.trimf(x, [1, 4, 7]) # Triangular membership function for fuzzy set B
8
9 # Create fuzzy relations based on the Cartesian product of A and B
10 relation_max_min = np.maximum.outer(A, B) # Max-min composition
11 relation_max_product = np.multiply.outer(A, B) # Max-product composition
12

```



```

13 print("Fuzzy Relation (Max-Min Composition):\n", relation_max_min)
14 print("\nFuzzy Relation (Max-Product Composition):\n", relation_max_product)
15

```

```

Fuzzy Relation (Max-Min Composition):
[[0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]
 [0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]
 [0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]
 [0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]
 [0.33333333 0.33333333 0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.33333333 0.33333333 0.33333333 0.33333333]
 [0.66666667 0.66666667 0.66666667 0.66666667 1.      0.66666667
  0.66666667 0.66666667 0.66666667 0.66666667 0.66666667]
 [1.      1.      1.      1.      1.      1.
  1.      1.      1.      1.      1.      ]
 [0.66666667 0.66666667 0.66666667 0.66666667 1.      0.66666667
  0.66666667 0.66666667 0.66666667 0.66666667 0.66666667]
 [0.33333333 0.33333333 0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.33333333 0.33333333 0.33333333 0.33333333]
 [0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]
 [0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]]

```

```

Fuzzy Relation (Max-Product Composition):
[[0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]
 [0.      0.      0.11111111 0.22222222 0.33333333 0.22222222
  0.11111111 0.      0.      0.      0.      ]
 [0.      0.      0.22222222 0.44444444 0.66666667 0.44444444
  0.22222222 0.      0.      0.      0.      ]
 [0.      0.      0.33333333 0.66666667 1.      0.66666667
  0.33333333 0.      0.      0.      0.      ]
 [0.      0.      0.22222222 0.44444444 0.66666667 0.44444444
  0.22222222 0.      0.      0.      0.      ]
 [0.      0.      0.11111111 0.22222222 0.33333333 0.22222222
  0.11111111 0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.      ]]

```

QUESTION 7

Implementation of fuzzy distance metrics: i. Fuzzy Euclidian distance ii. Fuzzy hamming distance

```

1 import numpy as np
2
3 def fuzzy_euclidean_distance(A, B):
4     # A and B are dictionaries of linguistic terms and their membership values
5     euclidean_distance = np.sqrt(np.sum((A[key] - B[key])**2 for key in A.keys()))
6     return euclidean_distance
7
8 def fuzzy_hamming_distance(A, B):
9     # A and B are dictionaries of linguistic terms and their membership values
10    hamming_distance = np.mean([np.abs(A[key] - B[key]) for key in A.keys()])
11    return hamming_distance
12
13 # Example usage
14 # Two dictionaries A and B are defined to represent fuzzy sets.
15 # Each dictionary contains linguistic terms as keys (e.g., 'low', 'medium', 'high') and their corresponding membership values.
16
17 A = {'low': 0.2, 'medium': 0.5, 'high': 0.3}
18 B = {'low': 0.3, 'medium': 0.4, 'high': 0.3}
19
20 euclidean_distance = fuzzy_euclidean_distance(A, B)
21 hamming_distance = fuzzy_hamming_distance(A, B)
22
23 print("Fuzzy Euclidean Distance:", euclidean_distance)
24 print("Fuzzy Hamming Distance:", hamming_distance)
25

```

Fuzzy Euclidean Distance: 0.14142135623730948
Fuzzy Hamming Distance: 0.06666666666666665
<ipython-input-67-234e04d68b19>:5: DeprecationWarning: Calling np.sum(generator) is deprecated, and in the future will give a differ

```
euclidean_distance = np.sqrt(np.sum((A[key] - B[key])**2 for key in A.keys()))
```

QUESTION 8

Implementation of Mamdani inference system.

- The Mamdani inference system is one of the most widely used methods for fuzzy inference. It consists of three main components: fuzzification, rule evaluation, and defuzzification. Let's implement a simple Mamdani inference system step by step:

Fuzzification: Convert crisp inputs into fuzzy sets. **Rule Evaluation:** Apply fuzzy rules to determine the fuzzy output. **Defuzzification:** Convert fuzzy output into a crisp value.

```
1 # A simple Mamdani inference system for controlling fan speed based on temperature and humidity levels
2
3 import numpy as np
4 import skfuzzy as fuzz
5 from skfuzzy import control as ctrl
6
7 # Define input and output variables
8 temperature = ctrl.Antecedent(np.arange(0, 101, 1), 'temperature')
9 humidity = ctrl.Antecedent(np.arange(0, 101, 1), 'humidity')
10 fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan speed')
11
12 # Define fuzzy sets for input variables
13 temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 50])
14 temperature['warm'] = fuzz.trimf(temperature.universe, [0, 50, 100])
15 humidity['dry'] = fuzz.trimf(humidity.universe, [0, 0, 50])
16 humidity['wet'] = fuzz.trimf(humidity.universe, [0, 50, 100])
17
18 # Define fuzzy sets for output variable
19 fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0, 0, 50])
20 fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [0, 50, 100])
21 fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50, 100, 100])
22
23 # Define fuzzy rules
24 rule1 = ctrl.Rule(temperature['cold'] & humidity['dry'], fan_speed['low'])
25 rule2 = ctrl.Rule(temperature['warm'] | humidity['wet'], fan_speed['high'])
26
27 # Create control system
28 fan_speed_ctrl = ctrl.ControlSystem([rule1, rule2])
29
30 # Create simulation
31 fan_speed_sim = ctrl.ControlSystemSimulation(fan_speed_ctrl)
32
33 # Set inputs
34 fan_speed_sim.input['temperature'] = 30
35 fan_speed_sim.input['humidity'] = 70
36
37 # Compute output
38 fan_speed_sim.compute()
39
40 # Print output
41 print("Fan Speed:", fan_speed_sim.output['fan speed'])
42
```

Fan Speed: 81.42857142857139

QUESTION 9

Implement a program that performs fuzzy clustering on a set of data points and outputs the membership degrees for each data point in each cluster.

Fuzzy clustering is a technique used to assign membership degrees to data points in multiple clusters. One popular method for fuzzy clustering is the Fuzzy C-Means (FCM) algorithm

```

1 import numpy as np
2 import skfuzzy as fuzz
3
4 # Generate sample data
5 np.random.seed(42)
6 data = np.random.randn(100, 2)
7
8 # Perform fuzzy clustering (FCM algorithm)
9 centers, u, _, _, _, _ = fuzz.cluster.cmeans(
10     data.T, 3, m=2, error=0.005, maxiter=1000
11 )
12
13 # data.T transposes the data to match the expected input format.
14 # 3 specifies the number of clusters.
15 # m=2 sets the fuzziness parameter.
16 # error and maxiter are stopping criteria for the algorithm.
17
18
19 # Output membership degrees for each data point in each cluster
20 print("Membership degrees:")
21 print(u)
22 #u: A matrix where each row represents a data point, and each column represents the membership degree of that data point to each cluster.
23 #We print the membership degrees (u) for each data point in each cluster.

```

Membership degrees:

```

[[5.27217810e-01 4.82916479e-02 3.39511520e-01 2.16324636e-01
 9.47691322e-02 3.26388325e-01 7.77759822e-01 1.81236231e-01
 2.72193799e-02 5.08252714e-01 4.88279141e-01 8.87675928e-01
 6.07940472e-02 4.39673700e-02 1.28623735e-01 1.09931842e-01
 9.39948350e-01 8.64524076e-01 7.65052399e-01 5.06386050e-02
 3.08786227e-01 4.75507819e-01 2.05099364e-01 8.36711462e-02
 8.18315807e-01 7.31093453e-01 7.93854714e-02 9.66586566e-02
 6.55824565e-02 7.12089163e-04 1.45838185e-01 3.97113086e-01
 5.25527155e-02 3.66361078e-02 9.16201753e-01 3.77863665e-02
 5.48483101e-02 1.82346028e-01 5.90507575e-01 7.48952918e-01
 1.40081095e-01 5.76905046e-01 1.65989852e-01 2.41307894e-01
 8.68666481e-02 1.52973407e-02 1.04958482e-01 7.25744519e-01
 2.19898981e-01 4.94018415e-01 1.24155390e-01 6.68019395e-01
 1.37833648e-01 3.62839603e-01 4.48755931e-01 1.38719834e-01
 1.21673667e-01 1.55173361e-01 9.18776328e-01 1.45573853e-01
 8.72377879e-01 7.05940277e-01 1.04879718e-01 1.64316331e-01
 7.80087961e-01 8.41345106e-02 5.08797639e-02 1.17615109e-01
 8.03512546e-02 8.66237586e-01 1.67152653e-02 9.09213160e-02
 2.05679948e-02 3.96738434e-01 2.13415560e-01 1.71393909e-01
 3.34158200e-02 9.51829445e-01 2.99856954e-01 7.84522537e-02
 8.67742481e-02 7.31737238e-01 2.11502092e-01 9.22869121e-02
 7.26054203e-01 3.17175711e-01 1.61134496e-01 1.31385140e-02
 4.22560233e-02 1.40969755e-01 9.22106145e-01 5.24092574e-02
 8.51714553e-02 4.72597484e-01 5.43245985e-01 8.87077771e-02
 9.55679355e-01 1.51887273e-01 5.70010708e-03 9.52771364e-01]
[1.96231580e-01 6.63855938e-02 5.23585598e-01 1.53468301e-01
 6.05443492e-01 5.85766731e-01 1.40179889e-01 7.08666011e-01
 9.34740757e-01 3.84619051e-01 1.72246034e-01 7.48995167e-02
 8.71499069e-01 8.92882481e-01 8.12763343e-01 2.80026766e-01
 4.17667445e-02 7.31954845e-02 1.48786747e-01 8.92665964e-01
 1.73565259e-01 3.85367288e-01 6.94852705e-01 3.10962992e-01
 1.12885770e-01 1.47270663e-01 6.98393439e-01 8.93791876e-02
 9.00472741e-01 1.12003125e-03 7.75644255e-01 4.93465545e-01
 6.39370872e-02 9.00467857e-02 4.86118012e-02 6.15340683e-02
 1.15058179e-01 5.78946063e-01 2.58454186e-01 1.61823234e-01
 4.96343347e-01 1.62784727e-01 7.72154543e-01 1.50030746e-01
 6.73187839e-01 3.09260071e-02 8.47067370e-01 2.00301630e-01
 2.24278585e-01 3.34514352e-01 8.01036373e-01 2.63129930e-01
 4.19828807e-01 1.83862310e-01 2.63949573e-01 7.35053531e-01
 2.09846960e-01 4.98656841e-01 5.64689931e-02 1.15762626e-01
 6.58629767e-02 1.44244560e-01 1.53667806e-01 7.69620700e-01
 1.43937924e-01 8.34131839e-01 8.63330323e-01 3.92626807e-01
 8.80370678e-01 7.25390168e-02 2.98643783e-02 8.11563889e-01
 3.30684935e-02 4.84761399e-01 1.69873879e-01 2.08590775e-01
 9.20182938e-01 2.92881773e-02 1.77760767e-01 7.71287128e-01
 6.89056843e-01 1.21783353e-01 1.39991846e-01 1.21909273e-01
 2.11626988e-01 5.95318107e-01 3.84855925e-01 2.10237415e-02
 8.76486777e-02 2.55644770e-01 4.21200440e-02 8.58663714e-01
 2.84464170e-01 2.09273627e-01 3.50023916e-01 3.84857548e-01
 2.85846472e-02 2.19578417e-01 9.87826234e-01 3.19892555e-02]
[2.76550610e-01 8.85322758e-01 1.36902882e-01 6.30207063e-01
 2.99787376e-01 8.78449443e-02 8.20602894e-02 1.10097757e-01
 3.80398627e-02 1.07128235e-01 3.39474825e-01 3.74245554e-02
 6.77068836e-02 6.31501495e-02 5.86129226e-02 6.10041392e-01
 1.82849050e-02 6.22804396e-02 8.61608544e-02 5.66954307e-02
 5.17648514e-01 1.39124892e-01 1.00047931e-01 6.05365862e-01
 6.87984231e-02 1.21635884e-01 2.22221090e-01 8.13962156e-01

```

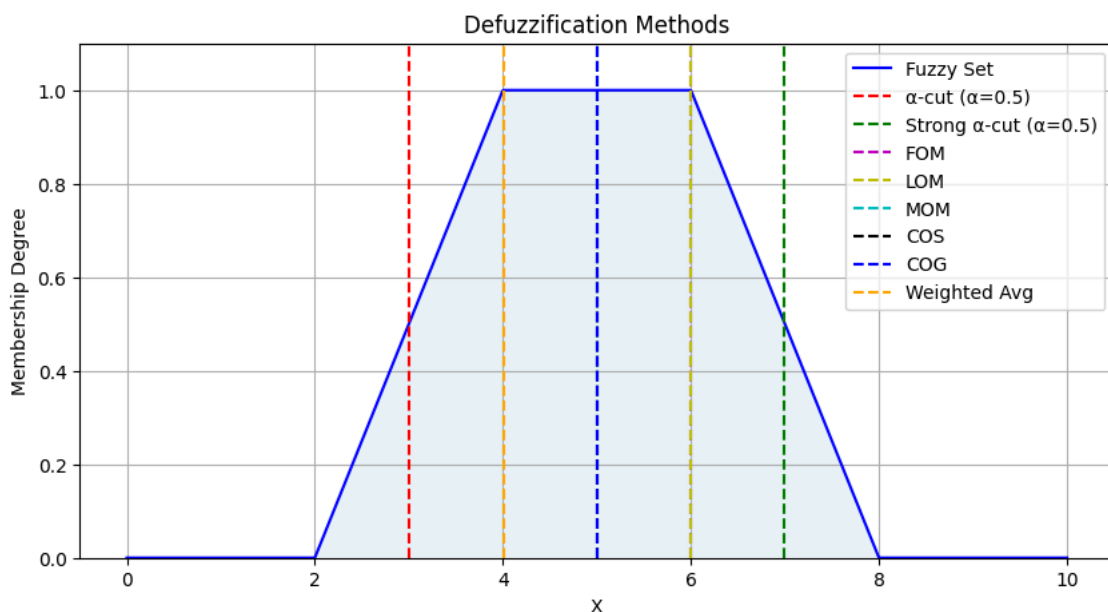
** QUESTION 10**

Implement and plot different defuzzification methods: i. α -cut and strong α -cut ii. Maxima method: a. First of Maxima (FOM) b. Last of Maxima (LOM) c. Mean of Maxima (MOM) iii. Centroid method: a. Center of Sums (COS) b. Center of gravity (COG) / Centroid of Area (COA) Method c. Weighted average method

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import skfuzzy as fuzz
4
5 # Generate sample fuzzy set
6 x = np.linspace(0, 10, 1000)
7 mfx = fuzz.trapmf(x, [2, 4, 6, 8]) # Sample fuzzy set (trapezoidal shape)
8
9 # Plot the sample fuzzy set
10 plt.figure(figsize=(10, 5))
11 plt.plot(x, mfx, 'b', linewidth=1.5, label='Fuzzy Set')
12 plt.fill_between(x, 0, mfx, alpha=0.1)
13
14 # Defuzzification methods
15 #  $\alpha$ -cut and strong  $\alpha$ -cut (calculate manually)
16 alpha = 0.5
17 alpha_cut = np.min(x[mfx >= alpha]) #  $\alpha$ -cut for  $\alpha=0.5$ 
18 strong_alpha_cut = np.max(x[mfx >= alpha]) # Strong  $\alpha$ -cut for  $\alpha=0.5$ 
19
20 # Maxima methods
21 fom = fuzz.defuzz(x, mfx, 'lom') # First of Maxima
22 lom = fuzz.defuzz(x, mfx, 'lom') # Last of Maxima
23 mom = fuzz.defuzz(x, mfx, 'centroid') # Mean of Maxima
24
25 # Centroid methods
26 cos = fuzz.defuzz(x, mfx, 'centroid') # Center of Sums
27 cog = fuzz.defuzz(x, mfx, 'centroid') # Center of gravity / Centroid of Area
28 weighted_avg = fuzz.defuzz(x, mfx, 'som') # Weighted average method
29
30 # Plot defuzzified values
31 plt.axvline(x=alpha_cut, color='r', linestyle='--', label=' $\alpha$ -cut ( $\alpha=0.5$ )')
32 plt.axvline(x=strong_alpha_cut, color='g', linestyle='--', label='Strong  $\alpha$ -cut ( $\alpha=0.5$ )')
33 plt.axvline(x=fom, color='m', linestyle='--', label='FOM')
34 plt.axvline(x=lom, color='y', linestyle='--', label='LOM')
35 plt.axvline(x=mom, color='c', linestyle='--', label='MOM')
36 plt.axvline(x=cos, color='k', linestyle='--', label='COS')
37 plt.axvline(x=cog, color='b', linestyle='--', label='COG')
38 plt.axvline(x=weighted_avg, color='orange', linestyle='--', label='Weighted Avg')
39
40 # Additional plotting configurations
41 plt.ylim(0, 1.1)
42 plt.title('Defuzzification Methods')
43 plt.xlabel('X')
44 plt.ylabel('Membership Degree')
45 plt.legend()
46 plt.grid(True)
47 plt.show()
48

```



Implement the AND, OR and XOR function using single layer and multilayer neural network.

```
1 # Implementation of the single-layer neural network:
2
3 import numpy as np
4 import tensorflow as tf
5
6 # Define the inputs and outputs for AND, OR, and XOR functions
7 inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
8 and_outputs = np.array([[0], [0], [0], [1]], dtype=np.float32)
9 or_outputs = np.array([[0], [1], [1], [1]], dtype=np.float32)
10 xor_outputs = np.array([[0], [1], [1], [0]], dtype=np.float32)
11
12 # Define the single-layer neural network
13 def single_layer_nn(inputs, outputs):
14     model = tf.keras.Sequential([
15         tf.keras.layers.Dense(units=1, input_shape=(2,), activation='sigmoid')
16     ])
17     model.compile(optimizer='adam', loss='mse')
18     model.fit(inputs, outputs, epochs=1000, verbose=0)
```